

Квантовый апокалипсис отменяется: гайд по переходу на Post-Quantum Cryptography (PQC)

Если ты пишешь бэкенд, фронтенд или мобилку — скорее всего, ты уже используешь шифрование. HTTPS, JWT, TLS — всё это работает на математике, которую квантовый компьютер сломает. Не сразу, но уже понятно когда.

Это не повод паниковать. Это повод разобраться — что именно сломается, что нет, и что нужно поменять в своём коде.

После этого материала ты будешь знать:

- Почему RSA и ECC обречены, а AES-256 — нет
- Что такое атака «собирай сейчас — читай потом» и затрагивает ли она твой сервис
- Какие новые алгоритмы пришли на замену и чем они отличаются друг от друга
- Почему переход на PQC может сломать соединения — и как этого избежать
- Как написать гибридную защиту, которая работает уже сегодня

Оглавление

- [Что происходит и почему сейчас](#)
- [История про Артёма — как выглядит атака](#)
- [Почему одни алгоритмы сломаются, а другие нет](#)
- [Новые алгоритмы: что выбрать и для чего](#)
- [Подводные камни при внедрении](#)
- [Как защититься уже сегодня: гибридный подход](#)
- [Реальный кейс: как это сделал Signal](#)
- [Тест](#)
- [Задание: найди и исправь уязвимости в конфиге TLS](#)

Что происходит и почему сейчас

В августе 2024 года NIST (американский институт стандартов) опубликовал новые стандарты шифрования — специально разработанные против квантовых компьютеров. Apple уже защитила iMessage, Google обновил Chrome, Signal переписал протокол обмена ключами.

Квантового компьютера, который ломает шифрование, ещё нет. Но есть три причины начинать прямо сейчас:

Причина 1: данные крадут уже сегодня. Зашифрованный трафик можно перехватить и хранить — а расшифровать потом, когда появится нужный компьютер. Если твои данные должны быть секретными ещё лет через пять — они уже под угрозой.

Причина 2: миграция занимает годы. Обновить один сервис — легко. Обновить все сервисы, партнёрские интеграции, мобильные клиенты, IoT-устройства в поле — это большой проект.

Причина 3: прогресс ускоряется. В мае 2025 года вышла оптимизация алгоритма Шора, которая снизила требования к квантовому железу в 20 раз. Прогноз сдвинулся с «никогда» на начало 2030-х.

Коротко: переход на PQC — это не «когда-нибудь». Это инфраструктурный проект, который нужно начинать сейчас.

История про Артёма

Давай сначала посмотрим, как атака выглядит на практике. Это поможет понять, что именно мы защищаем.

Знакомьтесь: Артём

Артём — аналитик данных в небольшой компании. Днём он строит графики в Excel и пьёт третий кофе, а по ночам... по ночам он работает над долгосрочным инвестиционным проектом.

Артём перехватывает корпоративный трафик крупного банка и аккуратно складывает его на жёсткий диск.

Он не спешит. У него есть план.

Коллеги называют его «самым терпеливым человеком в комнате». Артём только улыбается и запускает очередной дамп TLS-сессий в архив.

Шаг 1. Сбор урожая

Место действия: арендованный сервер. 2025 год.

Артём настраивает перехват трафика через подконтрольный ему узел. Банк использует TLS 1.3 с RSA-2048 и ECDH на кривой P-256. Всё по учебнику. Всё надёжно.

— Сейчас — да, — бормочет Артём и запускает сниффер.

```
[CAPTURE] TLS session dump → archive_2025_01.enc [4.2 GB]
[CAPTURE] TLS session dump → archive_2025_02.enc [3.8 GB]
[CAPTURE] TLS session dump → archive_2025_03.enc [5.1 GB]
...
```

Трафик зашифрован. Артём это знает. Его это не беспокоит.

Что происходит: злоумышленник записывает TLS-сессии целиком — включая ClientHello и ServerHello с публичными ключами. Расшифровать сейчас невозможно, но сами байты никуда не деваются.

Шаг 2. Холодное хранение

За месяц набирается ~60 ГБ: транзакции, внутренние API-вызовы, переписка систем.

```
> tar -czf archive_2025_01.enc.gz ./harvest/2025-01/*
> rclone copy archive_2025_01.enc.gz cold-storage:/bank_harvest/
> du -sh cold-storage:/bank_harvest/
61.4 GB total> echo "Стоимость хранения: 1.23 в месяц. Дешевле подписки на стриминг."
```

Артём закрывает ноутбук и идёт спать. Завтра — обычный рабочий день.

Шаг 3. Ожидание: 2025 → 2031

Артём продолжает работать аналитиком. Читает новости о квантовых компьютерах. Следит за IBM и Google.

Однажды утром он видит заголовок:

«Google объявляет о первом CRQC: RSA-2048 взломан за 4 часа»

Артём медленно допивает кофе и открывает терминал.

Шаг 4. Час расплаты

2031 год. Артём достаёт архивы шестилетней давности.

```
> quantum_shor_client --algorithm ECDH-P256 --target archive_2025_01.enc
[INFO] Загружено 61400 TLS-сессий.
[INFO] Восстановление session key для сессии 2025-01-15 10:23:17...
[████████████████████] 100%
[SUCCESS] Session key recovered: a3f9e4b7c812d5467f1a9b3c5d7e8f2a

> decrypt_tls_session --key a3f9e4b7c812d5467f1a9b3c5d7e8f2a --input
harvest_20250115.enc
[SUCCESS] → decrypted_20250115.json

{
  "transaction_id": "TXN-20250115-009823",
  "from_account": "****4471",
  "to_account": "****8832",
```

```
"amount": 2850000,  
"currency": "RUB",  
"auth_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."  
}
```

Шесть лет ожидания. Один скрипт. Полный архив транзакций.

— Терпение, — говорит он себе. — Терпение и труд всё перетрут.

Что пошло не так?

Банк сделал три ошибки:

1. **Использовал ECDH P-256** — эллиптические кривые уязвимы для алгоритма Шора. Ключ короткий, кубитов нужно меньше — ECC ломается первой (об этом дальше).
2. **Не внедрил гибридный обмен ключами** — если бы рядом с ECDH работал ML-KEM (Kyber), Артёму понадобился бы и квантовый компьютер, и уязвимость в Kyber одновременно.
3. **Откладывал миграцию** — данные были украдены раньше, чем угроза стала реальной.

Мог ли Артём потерпеть неудачу? Да. Достаточно было одной строки в конфиге — гибридный обмен ключами:

```
MasterSecret = KDF(SharedSecret_ECDH || SharedSecret_Kyber)
```

Даже взломав ECDH квантовым компьютером, Артём получил бы только половину входных данных для финального ключа. Без Kyber-части — ничего.

Теорема Моска: когда именно горит?

Исследователь Микеле Моска сформулировал простое правило риска:



Вот калькулятор для самопроверки:

```
import datetime

def check_quantum_risk(data_type: str, shelf_life_years: int,
migration_estimate_years: int):
    """
    Проверяет, успеваешь ли ты с миграцией.
    shelf_life_years — X: сколько лет данные должны быть секретны
    migration_estimate_years — Y: сколько займёт внедрение PQC
    """
    YEAR_CRQC_ARRIVAL = 2032 # консервативный прогноз
    current_year = datetime.date.today().year
    z = YEAR_CRQC_ARRIVAL - current_year # сколько лет осталось
    total = shelf_life_years + migration_estimate_years

    print(f"\n--- {data_type} ---")
    if total > z:
        print(f"🔴 РИСК. Ты опаздываешь на {total - z} лет.")
    else:
        print(f"🟢 Запас: {z - total} лет.")
    # Попробуй со своими данными:
```

```
check_quantum_risk("Сессионные токены (живут 24ч)", shelf_life_years=0,
migration_estimate_years=2)
check_quantum_risk("История транзакций (храним 7 лет)",
shelf_life_years=7, migration_estimate_years=3)
check_quantum_risk("Медицинские данные (бессрочно)", shelf_life_years=20,
migration_estimate_years=4)
```

Запусти это с параметрами своего сервиса — и сразу поймёшь, насколько срочно.

Почему одни алгоритмы сломаются, а другие нет

RSA и ECC: почему они обречены

Вся асимметричная криптография держится на «задачах с потайным ходом» — математике, которую легко решить в одну сторону и почти невозможно в обратную.

- **RSA** — на факторизации: перемножить два простых числа легко, разложить обратно — нет.
- **ECC и ECDH** — на дискретном логарифмировании в группе точек эллиптической кривой.

Алгоритм Шора (1994) умеет находить период функции за полиномиальное время — и обе задачи становятся тривиальными для квантового компьютера.

Неочевидный факт про ECC: мы привыкли считать его «надёжнее» RSA — ключ ECC-256 по классической стойкости эквивалентен RSA-3072, но компактнее. В квантовом мире компактность оборачивается уязвимостью: короткий ключ требует меньше кубитов для взлома.

Алгоритм	Логических кубитов для взлома
RSA-2048	~4098
ECDSA-256	~2330

Подводные камни при внедрении

Проблема 1: новые ключи не влезают в пакеты

Ключ X25519 — 32 байта. Ключ ML-KEM-768 — 1184 байта. Разница в 37 раз.

Стандартный MTU большинства сетей — **1500 байт**. Классический `ClientHello` в TLS занимает 300–500 байт. После добавления Kyber он вырастает примерно до 1600–1800 байт и перестаёт влезать в один пакет.

Что происходит:

1. TCP разбивает `ClientHello` на несколько пакетов — сервер ждёт все.
2. На нестабильных мобильных сетях это добавляет 20–40% ко времени установки соединения.
3. **Главная ловушка — Middlebox Ossification:** старые корпоративные фаерволы и DPI-системы не распознают фрагментированный `ClientHello` как легитимный TLS и просто дропают пакет. По данным Cloudflare и Google, ~1–2% соединений в интернете ломаются именно из-за этого.

Упрощённая симуляция: что происходит с пакетом на разных типах сети

```
class NetworkPath:
def __init__(self, mtu: int = 1500, drop_fragmented: bool = False):
self.mtu = mtu
self.drop_fragmented = drop_fragmented # True = старый корпоративный
    фаервол

def transmit(self, data_bytes: bytes) -> bool:
packet_size = len(data_bytes)
effective_mtu = self.mtu - 40 # вычитаем IP(20) + TCP(20) заголовки

if packet_size <= effective_mtu:
print(f"✅ Пакет {packet_size}б прошёл целиком.")return True

fragments = (packet_size + effective_mtu - 1) // effective_mtu
print(f"⚠ Пакет {packet_size}б > MTU. Разбито на {fragments} фрагмента.")
if self.drop_fragmented:
# Вот здесь у пользователя зависает загрузка страницы
print("🚫 Старый фаервол дропнул соединение (Middlebox
    Ossification).")return False

print("✅ Фрагменты доставлены (но latency вырос).")return True
```

```
classic_hello = b'\x00' * 400 # обычный ClientHello
pqc_hello = b'\x00' * 1700 # ClientHello с гибридным Kyber-768

print("=== Современный роутер ===")
NetworkPath(mtu=1500, drop_fragmented=False).transmit(pqc_hello)

print("\n=== Корпоративный Legacy Firewall ===")
NetworkPath(mtu=1500, drop_fragmented=True).transmit(pqc_hello)
```

Проблема 2: CPU быстро, сеть медленно

Kyber на современном процессоре работает быстрее X25519 — матричные операции и хеширование CPU любит больше, чем математику эллиптических кривых.

Но ты выигрываешь микросекунды на CPU и теряешь миллисекунды на передаче лишних килобайтов. Для сервера в дата-центре разница незаметна. Для IoT-устройства на слабом канале — это батарея, которая кончается раньше.

Как защититься уже сегодня: гибридный подход

Зачем гибрид, а не просто «переключиться на Kyber»?

Решётки изучаются 20–30 лет, тогда как факторизация — столетиями. Kyber может содержать математическую ошибку, которую ещё не нашли. Если переключиться только на него — а потом найдут баг — потеряем всё.

Решение: использовать оба алгоритма одновременно, «ремень и подтяжки»:

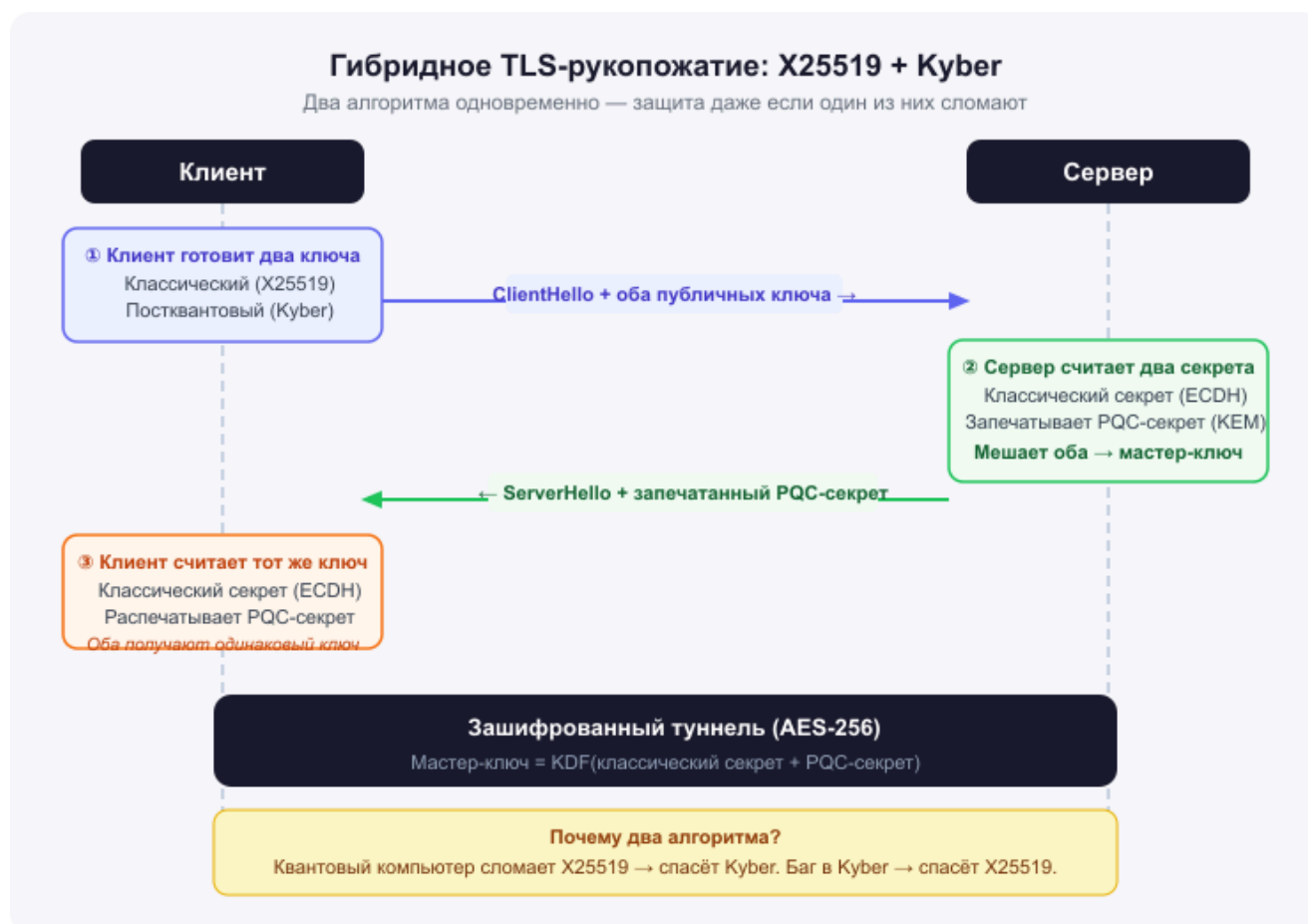
```
MasterSecret = KDF(SharedSecret_ECDH || SharedSecret_Kyber)
```

Чтобы взломать это соединение, атакующему нужно одновременно:

- Иметь квантовый компьютер (чтобы взломать ECDH), **И**
- Знать математическую уязвимость в Kyber.

Это нетривиальное требование. Apple, Google и Signal выбрали именно этот подход.

Схема TLS-рукопожатия с гибридным обменом



Клиент генерирует два `keypair` — классический X25519 и постквантовый ML-KEM — и отправляет оба публичных ключа в `ClientHello`. Сервер вычисляет классический ECDH-секрет и инкапсулирует PQC-секрет. Финальный мастер-ключ получается из обоих через KDF.

Матрица безопасности гибридного подхода

Сценарий атаки	ECDH (X25519)	ML-KEM (Kyber)	Результат (Гибрид)
Классический взлом (сегодня)	✓ Стойкий	✓ Стойкий	✓ БЕЗОПАСНО
Математический провал PQC (найден баг в Kyber)	✓ Стойкий	✗ Взломан	✓ БЕЗОПАСНО (спасает ECDH)
Квантовый компьютер (2030+)	✗ Взломан	✓ Стойкий	✓ БЕЗОПАСНО (спасает Kyber)

Практическое задание: Реализация гибридного KDF

Давайте смоделируем логику гибридного вывода ключей.

```

import hashlib
import hmac
import secrets

def hkdf_extract(salt: bytes | None, input_key_material: bytes) -> bytes:
    """
    Упрощённый HKDF-Extract (RFC 5869).
    Смешивает несколько секретов в один надёжный ключ.
    В реальном коде используй полный HKDF (Extract + Expand) из
    cryptography.hazmat.
    """
    if salt is None:
        salt = bytes(hashlib.sha256().digest_size) # нулевой соль по RFC
    return hmac.new(salt, input_key_material, hashlib.sha256).digest()

def derive_hybrid_secret(
    classic_secret: bytes,
    pqc_secret: bytes,
    context_info: bytes = b"tls13_hybrid_handshake" # метка протокола – важна
    для изоляции ключей
) -> bytes:
    """
    Финальный мастер-ключ = KDF(classic_secret || pqc_secret || context).
    Порядок конкатенации должен быть зафиксирован в протоколе – не меняй его.
    """
    combined = classic_secret + pqc_secret + context_info
    return hkdf_extract(salt=None, input_key_material=combined)

if __name__ == "__main__":
    # Имитируем нормальное рукопожатие
    alice_ecdh = secrets.token_bytes(32) # SharedSecret от X25519
    alice_kyber = secrets.token_bytes(32) # SharedSecret от ML-KEM

    key_safe = derive_hybrid_secret(alice_ecdh, alice_kyber)
    print(f"Ключ сессии: {key_safe.hex()[:16]}...")

    # Сценарий «Q-Day»: квантовый компьютер взломал ECDH
    # Атакующий знает alice_ecdh, но не знает alice_kyber
    attacker_guess = derive_hybrid_secret(alice_ecdh, secrets.token_bytes(32))
    print(
        "✅ ЗАЩИТА РАБОТАЕТ: взломав ECDH, ключ не угадать." if attacker_guess !=

```

```
key_safe
else "❌ Ключ угадан — что-то пошло не так!")

# Сценарий «Math Fail»: найден баг в Kyber
# Атакующий знает alice_kyber, но не знает alice_ecdh
attacker_guess_2 = derive_hybrid_secret(secrets.token_bytes(32),
alice_kyber)
print(
"✅ ЗАЩИТА РАБОТАЕТ: взломав Kyber, ключ не угадать." if attacker_guess_2
!= key_safe
else "❌ Ключ угадан — что-то пошло не так!")
```

Вывод: Гибридная схема — это страховка. Внедряя PQC сегодня, мы не отказываемся от проверенной классики, а наслаиваем новую защиту поверх неё. Это единственный способ безопасно пережить переходный период.

Реальный кейс: как это сделал Signal

Signal в 2023–2024 гг. первым среди массовых мессенджеров внедрил постквантовую защиту — протокол **PQXDH** (Post-Quantum Extended Diffie-Hellman).

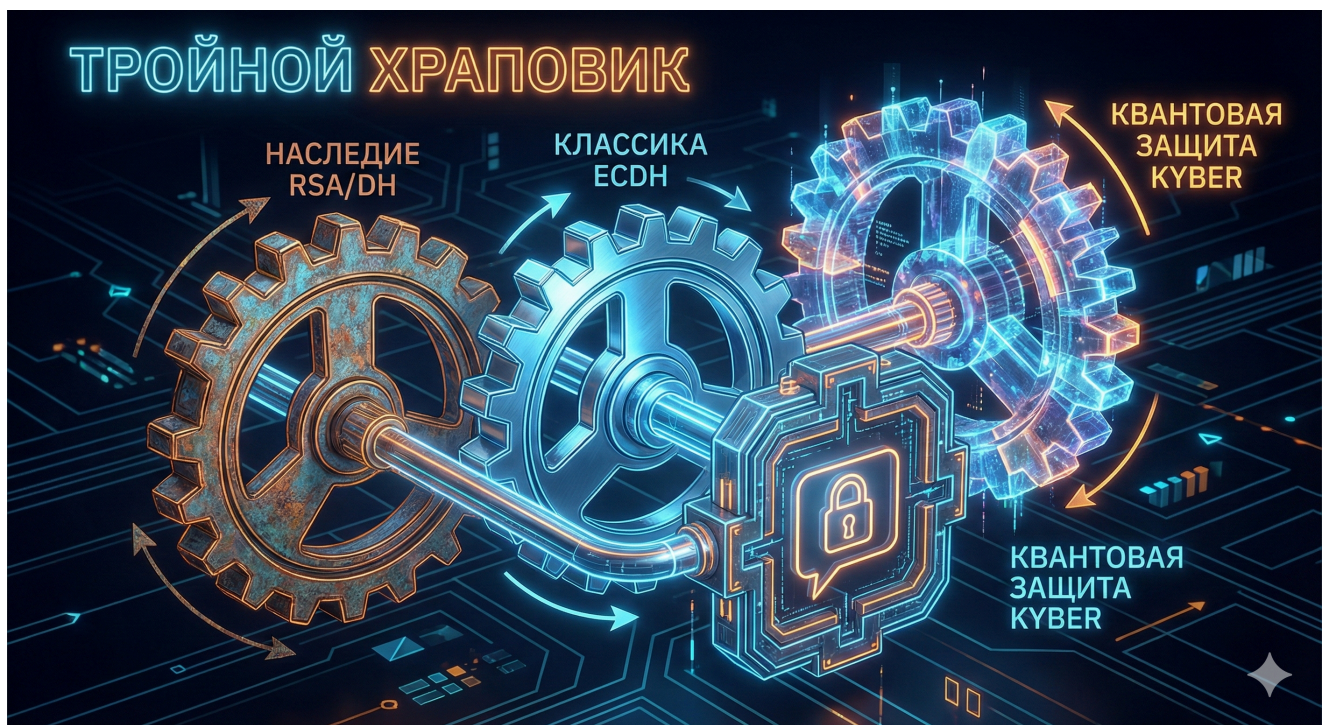
Трудность была в том, что Signal — **асинхронный** мессенджер. В TLS оба участника онлайн, и рукопожатие происходит в реальном времени. В Signal Боб может три дня не открывать приложение — а Алиса хочет написать ему прямо сейчас. Поэтому ключи Боба лежат на сервере заранее (Pre-Keys).

Что изменилось: к обычному набору Pre-Keys (Curve25519) Signal добавил постквантовый ключ Kyber-1024. Когда Алиса пишет Бобу:

1. Скачивает его классический ключ (Curve25519) и PQC-ключ (Kyber-1024).
2. Генерирует оба секрета — через ECDH и через KEM.
3. Смешивает их в финальный ключ шифрования.

Дополнительно Signal внедрил **Triple Ratchet** — к классическому «двойному храповику» добавился слой SPQR (Sparse Post-Quantum Ratchet). Kyber-ключи большие, поэтому их не обновляют при каждом сообщении — вместо этого «размазывают» по нескольким сообщениям через Erasure Codes.

Результат — **Post-Compromise Security**: если телефон взломали и украли ключи, то через несколько сообщений PQC Ratchet сработает, канал обновится, и атакующий снова «ослепнет» — без квантового компьютера новые ключи не вскрыть.



Тест

Вопрос 1.

Нужно ли срочно менять AES-256 на что-то принципиально новое для защиты от квантовых компьютеров?

- А) Да, AES полностью взломан алгоритмом Шора.
- Б) Нет, AES-256 достаточно — алгоритм Гровера снижает стойкость вдвое (до 128 бит), что остаётся безопасным.
- В) Да, нужно переходить на AES-512.

Правильный ответ: Б.

AES-256 выживает. Гровер даёт квадратичное ускорение: 256 бит → 128 эффективных. Просто убедись, что не используешь AES-128 — его нужно обновить.

Вопрос 2.

Что произойдёт с TLS-рукопожатием, если размер `ClientHello` из-за PQC-ключей превысит MTU (1500 байт)?

- А) Ничего страшного, пакет просто фрагментируется на уровне TCP.
- Б) Соединение мгновенно разорвётся с ошибкой шифрования.
- В) Пакет фрагментируется, но старое сетевое оборудование (Middleboxes) может его дропнуть — соединение зависнет.

Правильный ответ: В.

Middlebox Ossification — реальная проблема. ~1–2% соединений в интернете ломаются именно из-за этого. Немного — пока не умножить на всю аудиторию.

Вопрос 3.

В чём главный смысл гибридного обмена ключами (X25519 + Kyber)?

- А) Использовать квантовый компьютер для генерации ключей.
- Б) Объединить классический и постквантовый алгоритм — данные защищены, даже если один из них окажется уязвимым.
- В) Использовать два разных постквантовых алгоритма для ускорения.

Правильный ответ: Б.

«Ремень и подтяжки»: если Kyber взломают — спасёт ECDH. Если ECDH взламывает квантовый компьютер — спасёт Kyber.

Вопрос 4.

Ты пишешь систему обновления прошивки для промышленных устройств (срок службы — 15 лет). Проверка подписи происходит редко — при каждом обновлении раз в полгода. Что выбрать?

- А) ML-DSA (Dilithium)
- Б) SLH-DSA (SPHINCS+)
- В) RSA-4096

Правильный ответ: Б.

SLH-DSA основан на хеш-функциях, а не решётках — максимально консервативный выбор для долгосрочного применения. Подпись большая, но раз в полгода это не проблема.

Вопрос 5.

Почему угроза HNDL актуальна сегодня, если квантового компьютера ещё нет?

- А) Потому что хакеры уже используют квантовые эмуляторы.
- Б) Потому что данные, перехваченные сегодня, можно расшифровать в будущем — когда квантовый компьютер появится.
- В) Это маркетинговый ход, угрозы не существует.

Правильный ответ: Б.

Артём не расшифровывал трафик в 2025-м — он только собирал. Расшифровка случилась в 2031-м. Момент кражи и момент прочтения разделены во времени.

Вопрос 6.

Какой из стандартов NIST предназначен для замены ECDH в TLS?

- А) ML-DSA
- Б) SLH-DSA
- В) ML-KEM (Kyber)

Правильный ответ: В.

ML-KEM — это Key Encapsulation Mechanism, замена для обмена ключами. ML-DSA и SLH-DSA — алгоритмы цифровой подписи.

Вопрос 7.

После включения гибридного TLS ты замечаешь рост времени загрузки у мобильных пользователей на медленных соединениях. В чём скорее всего причина?

- А) Высокая нагрузка на процессор при вычислении ключей.
- Б) Увеличение размера передаваемых данных (публичный ключ + шифротекст Kyber), что повышает latency.
- В) Телефоны не поддерживают новые алгоритмы.

Правильный ответ: Б.

Kyber на CPU работает даже быстрее X25519. Проблема — в сети: лишние ~1200 байт в `ClientHello` ощутимы на медленных каналах.

Задание: найди и исправь уязвимости в конфиге TLS

Контекст

Ты — разработчик в финтех-стартапе. Тебе поручили обновить конфигурацию TLS для микросервиса, обрабатывающего транзакции. Текущий конфиг уязвим для атаки HNDL.

Задача — перевести сервис на **гибридную схему** с поддержкой обратной совместимости, чтобы старые клиенты не отвалились.

Исходный код с уязвимостями

```
from hypothetical_ssl_lib import SSLContext, Protocols

def create_context():
    ctx = SSLContext(protocol=Protocols.TLSv1_3)

    # [УЯЗВИМОСТЬ 1]
    # Жёсткая привязка к классической эллиптической кривой secp256r1.
    # Когда появится квантовый компьютер — весь архивный трафик будет
    # расшифрован.
    ctx.set_key_exchange_group("secp256r1")

    # [УЯЗВИМОСТЬ 2]
    # AES-128: алгоритм Гровера снижает стойкость до 64 бит — небезопасно.
    ctx.set_ciphers("TLS_AES_128_GCM_SHA256")
```

```
return ctx
```

Задание

1. Замени группу обмена ключами на гибридную (X25519 + Kyber768).
2. Добавь fallback: если клиент не поддерживает PQC — сервер соглашается на X25519, но не на RSA.
3. Обнови симметричный шифр для устойчивости к алгоритму Гровера.

Подсказки:

- *Подсказка 1:* Для защиты от Гровера нужен ключ ≥ 256 бит.
- *Подсказка 2:* Метод принимает список; порядок = приоритет.
- *Подсказка 3:* Гибридная группа: "x25519_mlkem768" .

Решение

```
from hypothetical_ssl_lib import SSLContext, Protocols

def create_quantum_safe_context():
    ctx = SSLContext(protocol=Protocols.TLSv1_3)

    # Исправление 1: гибридный обмен ключами + fallback для старых клиентов.
    # Первый элемент — предпочтительный. Если клиент не знает x25519_mlkem768,
    # сервер автоматически перейдёт к x25519 — соединение не сломается.
    ctx.set_key_exchange_groups([
        "x25519_mlkem768", # гибрид — предпочтительный
        "x25519", # fallback для клиентов без PQC-поддержки
    ])

    # Исправление 2: 256-битный ключ устойчив к Гроверу (128 эффективных бит).
    # ChaCha20 добавлен как альтернатива для мобильных устройств без
    аппаратного AES.
    ctx.set_ciphers([
        "TLS_AES_256_GCM_SHA384",
        "TLS_CHACHA20_POLY1305_SHA256",
    ])

    return ctx
```

Почему это работает

Гибридный обмен: X25519 страхует от математических атак на Kyber, а Kyber страхует от квантового взлома X25519. Ни один из сценариев атаки не компрометирует оба алгоритма одновременно.

Fallback по приоритету: сервер предлагает гибрид первым. Старый клиент его не поймёт — согласует чистый X25519. Новый клиент получит полную PQC-защиту. Доступность сохраняется.

AES-256: единственный необходимый шаг для симметричной части — просто удвоить ключ.